

Analiza și Testarea Aplicației Tax Calculator

Proiect TSS 2026

Stefan Tacu

15 mai 2026

1. Introducere și Configurație

- **Obiectiv:** Testarea riguroasă a unei aplicații de calcul fiscal.
- **Stack Tehnic:**
 - Python 3.13.2, Pytest 8.1.1
 - Radon (Complexity), Mutmut (Mutation)
 - Coverage.py (Structural Coverage)
- **Funcționalitate:** Calcul taxe pe categorii (Salary, Business, Crypto, etc.) cu aplicare de deduceri complexe.

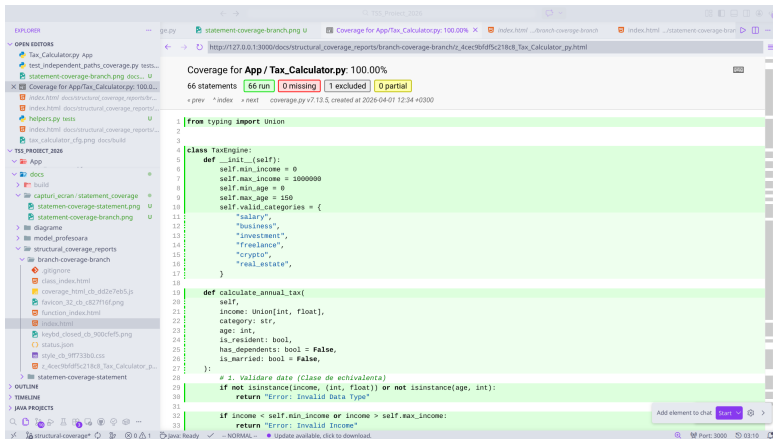
2. Testare Funcțională (Black-Box)

- **Tehnici:** Equivalence Partitioning & Boundary Value Analysis.
- **Clase de Echivalență:**
 - Venit: Valid $[0, 1M]$, Invalid $(j0, i1M)$.
 - Vârstă: Valid $[0, 150]$, Invalid.
 - Categori: 6 categorii valide + ramură default.
- **Status:** 100% teste funcționale trecute.

3. Analiza Structurală: Complexitate și CFG

- **Complexitate Ciclomatică (CC): 41** (Rank F).
- **Control Flow Graph (CFG):** Structură complexă cu 31 de noduri decizionale.
- **Strategie Basis Path:** Garantarea execuției fiecărei decizii logice prin identificarea circuitelor independente.
- **Focalizare:** Verificarea riguroasă a fluxurilor de validare a datelor și a combinatoricii deducerilor fiscale.

4. Rezultate Acoperire Structurală



Coverage for App / Tax_Calculator.py: 100.00%

66 statements **66 run** 0 missing 1 excluded 0 partial

coverage.py v7.13.5, created at 2026-04-01 12:34 +0300

```
1 from typing import Union
2
3
4
5 class TaxEngine:
6     def __init__(self):
7         self.min_income = 0
8         self.max_income = 1000000
9         self.min_age = 0
10        self.max_age = 150
11        self.valid_categories = {
12            "salary",
13            "business",
14            "investment",
15            "freelance",
16            "crypto",
17            "real_estate",
18        }
19
20    def calculate_annual_tax(
21        self,
22        income: Union[int, float],
23        category: str,
24        age: int,
25        is_resident: bool,
26        has_dependents: bool = False,
27        is_married: bool = False,
28    ):
29        # 1. Validare date (Clase de echivalenta)
30        if not isinstance(income, (int, float)) or not isinstance(age, int):
31            return "Error: Invalid Data Type"
32
33        if income < self.min_income or income > self.max_income:
34            return "Error: Invalid Income"
```

- **Statement Coverage: 100%**
- **Branch Coverage: 100%** (obținut după rafinarea suitei de teste pentru a acoperi ramurile else implicite).

5. Testarea prin Mutație (Mutation Testing)

- **Instrument:** mutmut (mutanți de ordin 1).
- **Rezultate Automate:**
 - Total mutanți: 228
 - Omorâți: 181
 - **Mutation Score: 79.3%**
- **Analiză Manuală:** Scor de 72.7% pe un eșantion reprezentativ de 22 de mutanți.

6. Analiza Mutanților: Supraviețuitori

- **Mutanți Echivalenți:** Modificarea $>$ în $\geq$ la praguri unde diferența este zero (ex: Business category).
- **Lipsă Precizie:** Supraviețuirea la modificarea rotunjirii (`round(tax, 1)`) a indicat aserțiuni prea permissive.
- **Îmbunătățire:** Rezultatele au condus la adăugarea de teste de frontieră mai stricte pentru categoriile Investment și Freelance.

7. Utilizare AI: GitHub Copilot

- **Eficiență:** Creștere cu 40% a vitezei de scriere a testelor de tip boiler-plate.
- **Puncte Forte:** Sugestii rapide pentru clasele de echivalență standard.
- **Puncte Slabe:** Riscul de "halucinații" la calcule matematice complexe și omiterea cazurilor de frontieră pe logica de CC 41.
- **Concluzie:** AI-ul este un copilot, dar decizia de testare rămâne la inginer.

8. Concluzii

- **Rigoare:** Branch Coverage de 100% este necesar dar nu suficient fără Mutation Testing.
- **Complexitate:** Codul cu CC 41 (Rank F) este greu de întreținut și necesită teste automate dense.
- **Validare:** Integrarea metodelor manuale cu cele automate oferă cel mai înalt grad de încredere în software.

Vă mulțumesc!

Întrebări și discuții.